

PUSH · SECURITY REVIEW

Push Chain Migration.

A security review of Chain Migration.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Push	Chain Migration	Guardian	June 20 – June 30, 2025	July 7, 2025

§ 00

Contents.

29 pages — overview, confidence ranking, scope, 19 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 17 findings	P. 09
	Remediation Review · 2 findings	P. 26
§ 05	Disclaimer	P. 28
§ 06	About Guardian	P. 29

§ 01

Overview.

Guardian reviewed Chain Migration across 2 review rounds. The final report includes 12 Low, 7 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	pushchain/push-chain-migration
CLIENT	METHODOLOGY
Push	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Chain Migration	a056ca2ce60f
PREPARED BY	
Guardian	
AUDIT WINDOW	
June 20 – June 30, 2025	
FINAL REPORT	
July 7, 2025	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	12	0	4	0	8
Informational	7	0	3	0	4
All severities	19	0	7	0	12

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Chain Migration in `pushchain/push-chain-migration`. The scope includes 171 source lines across 2 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/MigrationRelease.sol	117	181
src/MigrationLocker.sol	54	93

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	Redundant Boolean Check	Superfluous Code	LOW	RESOLVED	p. 9
L-02	Unused State Variable	Superfluous Code	LOW	RESOLVED	p. 10
L-03	Redundant Leaf Derivation	Gas Optimization	LOW	RESOLVED	p. 11
L-04	Unnecessary Return Keyword	Superfluous Code	LOW	RESOLVED	p. 12
L-05	Flawed Contract Detection Logic May Limit UX	DoS	LOW	ACKNOWLEDGED	p. 13
L-06	Users May Lock Tokens In Wrong Contract	Validation	LOW	ACKNOWLEDGED	p. 14
L-07	Missing SafeERC20 Implementation	Best Practices	LOW	RESOLVED	p. 15
L-08	Missing New Epoch Event	Events	LOW	RESOLVED	p. 16
L-09	Locks May Not Be Included In Merkle Root	Warning	LOW	ACKNOWLEDGED	p. 17
L-10	Validation For Leaf Sum Matching	Best Practices	LOW	RESOLVED	p. 18
L-11	Missing Permit Functionality	Informational	LOW	RESOLVED	p. 19
I-01	Typo In releaseInstant Function	Best Practices	INFORMATIONAL	RESOLVED	p. 20
I-02	Release Time Not Emitted In Events	Events	INFORMATIONAL	RESOLVED	p. 21

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-03	Users May Double Claim	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 22
I-04	Locked Event Does Not Have Unique Identifiers	Documentation	INFORMATIONAL	RESOLVED	p. 23
I-05	Readme Addresses setToggleLock Function	Informational	INFORMATIONAL	RESOLVED	p. 24
I-06	Redundant Tree Construction In proofArray.js	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 25

REMEDIATION REVIEW

L-01	Unnecessary Indexed Event Params	Events	LOW	ACKNOWLEDGED	p. 26
I-01	Offchain Balance Checks Affected By Donations	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 27

LOW **L-01**

RESOLVED

Redundant Boolean Check

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/main/src/MigrationRelease.sol#L124C9-L124C41
REVIEW	Main Review

DESCRIPTION

The `releaseVested` function verifies if user already claimed by checking `claimedvested[leaf] = true`. As `claimedvested` is a boolean, there is no need to compare it to the `true` param.

RECOMMENDATION

Consider removing the comparison strict equality operator:

```
if (claimedvested[leaf] )
```

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/74a2f6e4beeb7012629df8a34c71eedd2023d77e>

LOW **L-02**

RESOLVED

Unused State Variable

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/audit-check-101/src/MigrationRelease.sol#L30
REVIEW	Main Review

DESCRIPTION

The `isClaimPaused` state variable is defined in `MigrationRelease.sol` but it is never used. Additionally, the `whenNotPaused` modifier already handles this feature.

RECOMMENDATION

Remove `isClaimPaused` state variable.

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/74a2f6e4beeb7012629df8a34c71eedd2023d77e>

LOW **L-03**

RESOLVED

Redundant Leaf Derivation

CATEGORY	Gas Optimization
LOCATION	https://github.com/push-protocol/push-chain-migration/blob/a056ca2ce60fa54f98fe021004a2676bac7a8c3f/src/MigrationRelease.sol#L99-L100
REVIEW	Main Review

DESCRIPTION

```
bytes32 leaf = keccak256(abi.encodePacked(_recipient, _amount,
_epoch));require(verifyAddress(_recipient, _amount, _epoch, _merkleProof) &
instantClaimTime[leaf] = 0,"Not Whitelisted or already Claimed");
```

The leaf is locally constructed using `_recipient`, `_amount`, and `_epoch`, then passed to `verifyAddress`, which re-computes the same hash again. This is redundant and introduces unnecessary computation.

RECOMMENDATION

Consider computing the `leaf` once and passing it directly to the Merkle verification.

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/74a2f6e4beeb7012629df8a34c71eedd2023d77e>

LOW **L-04**

RESOLVED

Unnecessary Return Keyword

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/main/src/MigrationRelease.sol#L166
REVIEW	Main Review

DESCRIPTION

The explicit `return` statement at the end of this conditional block is unnecessary, as the function execution will naturally return at this point.

```
if (_token = address(0)) {transferFunds(_to, _amount);return;}
```

RECOMMENDATION

Consider removing the explicit `return;`

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/74a2f6e4beeb7012629df8a34c71eedd2023d77e>

LOW **L-05**

ACKNOWLEDGED

Flawed Contract Detection Logic May Limit UX

CATEGORY	DoS
LOCATION	https://github.com/push-protocol/push-chain-migration/blob/a056ca2ce60fa54f98fe021004a2676bac7a8c3f/src/MigrationLocker.sol#L73-L76
REVIEW	Main Review

— DESCRIPTION

The Push team intends to prevent smart contracts from being recipient by checking the code size using the following logic:

```
uint256 codeLength;assembly {codeLength := extcodesize(_recipient) // @audit What is this codeLength check supposed to prevent?}
```

This check would block legitimate smart contract wallets (e.g., Gnosis Safe, ERC-4337 wallets) from interacting, which could limit usability.

— RECOMMENDATION

Consider removing the `extcodesize` check.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Users May Lock Tokens In Wrong Contract

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/audit-check-101/src/MigrationLocker.sol#L72>

REVIEW Main Review

DESCRIPTION

The `MigrationLocker` is an upgradeable contract. Although some safety checks were enforced to avoid users initializing the implementation, nothing prevents a user from locking PUSH tokens in this contract by mistake. This is possible as the `PUSH` token is a constant and the contract is unpaused by default.

RECOMMENDATION

Consider calling `_pause()` in the `constructor` so future calls to the `lock` function in the implementation contract reverts.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

RESOLVED

Missing SafeERC20 Implementation

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/audit-check-101/src/MigrationLocker.sol>

REVIEW Main Review

DESCRIPTION

The `MigrationRelease` uses `SafeERC20` in the transfer function for `recoverFunds`. However, the `MigrationLocker` also contains a `recoverFunds` function but the `SafeERC20` lib is not used.

RECOMMENDATION

Consider, using `SafeERC20` library in the `MigrationLocker` contract.

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/4c59cb2c9eb2b28a7d499e183c2cce97e8be6de5>

LOW **L-08**

RESOLVED

Missing New Epoch Event

CATEGORY Events

LOCATION <https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/audit-check-101/src/MigrationLocker.sol#L52>

REVIEW Main Review

DESCRIPTION

Owner will call `initiateNewEpoch` to start a new epoch for locking PUSH tokens. This is an important state change in the contract that should emit an event to notify off chain services.

RECOMMENDATION

Consider emitting an epoch change event in the `initiateNewEpoch` function.

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/f91208575671326eabd79c94663eaf306ece72f4>

LOW **L-09**

ACKNOWLEDGED

Locks May Not Be Included In Merkle Root

CATEGORY Warning

LOCATION <https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/audit-check-101/script/utils/fetchAndStoreEvents.js#L39>

REVIEW Main Review

DESCRIPTION

The `fetchAndStoreEvents` will fetch all `Locked` events between two blocks. This script is meant to be used once per epoch, at the end of each epoch.

If the locker contract is not paused before calling `initiateNewEpoch`, the following edge case might appear:

- epoch = 1, started at block 0
- Alice submits `lock` tx at block 100, Locked event uses epoch 1
- owner also submits `initiateNewEpoch` at block 100, epoch = 2
- team runs `fetchAndStoreEvents` filtering epoch 1 events.
- $\text{startBlock} = \text{epochStartBlock}(1) = 0$
- $\text{endBlock} = \text{epochStartBlock}(1 + 1) - 1 = 100 - 1 = 99$
- `locker.queryFilter` will not include Alice's lock event.

RECOMMENDATION

Consider checking blocks up to and including the next block epoch's start block in the `fetchAndStoreEvents` script.

RESOLUTION

Status: Acknowledged.

LOW **L-10**

RESOLVED

Validation For Leaf Sum Matching

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Currently, there is no validation in `fetchAndStoreEvents` to ensure that the sum of all leaves equals the actual amount locked on the contract for that epoch.

Theoretically, issues with the RPC (e.g., null values due to rate limits or internal errors) could result in the script omitting some lock events during Merkle root construction.

Since this is a one-way migration for the Push token itself, it is critical to include post-execution validations to ensure consistency with the on-chain state.

— RECOMMENDATION

Consider tracking the amount locked for each epoch on-chain. After constructing the Merkle tree off-chain, sum the leaves and compare the result with the on-chain locked amount to verify that the off-chain state has been correctly captured.

— RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/3b2e10fc00d38dab12a9679c3fa51f14f02ca386>

LOW **L-11**

RESOLVED

Missing Permit Functionality

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/audit-check-101/src/MigrationLocker.sol#L72>

REVIEW Main Review

DESCRIPTION

The `lock` functions enforces users to first approve their PUSH tokens to be spent by the `MigrationLocker` contract. However, the PUSH token has a `permit` functionality, which can't be used in the locker contract.

RECOMMENDATION

Consider adding a function to lock tokens with permit signature.

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/e6b9c4d4a294a969898326305a231ef93254fc91>

INFORMATIONAL **I-01**

RESOLVED

Typo In releaseInstant Function

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/main/src/MigrationRelease.sol#L102>

REVIEW Main Review

DESCRIPTION

`relaese` instead of `release`

RECOMMENDATION

Consider correcting it

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/74a2f6e4beeb7012629df8a34c71eedd2023d77e>

INFORMATIONAL I-02

RESOLVED

Release Time Not Emitted In Events

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/audit-check-101/src/MigrationRelease.sol#L120C14-L120C95
REVIEW	Main Review

DESCRIPTION

The `MigrationRelease` contract emits the `ReleasedInstant` and `ReleasedVested` when users claim their PUSH tokens. These events emit the recipient's address, amount released, and epoch.

However, the natspec incorrectly states `emits a ReleasedVested event with the recipient address, amount, and release time`, but the epoch value is emitted instead.

RECOMMENDATION

Update the natspec to describe the correct parameters emitted in the release events, as the release time or `block.timestamp` will be included in the events anyways.

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/74a2f6e4beeb7012629df8a34c71eedd2023d77e>

INFORMATIONAL **I-03**

ACKNOWLEDGED

Users May Double Claim

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

If a user locks a , claims a , and then locks b , the script currently tracks their total as $a + b$. This could allow them to claim twice in the same epoch.

— RECOMMENDATION

Beware that there should be only one merkle root update per epoch.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04

RESOLVED

Locked Event Does Not Have Unique Identifiers

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/main/src/MigrationLocker.sol#L71
REVIEW	Main Review

DESCRIPTION

The current natspec of the `lock` functions states:

Emits a Locked event with the recipient address, amount, and a unique identifier

However, this was an old version of the contract, where an `id` was used in the `Locked` event, instead of the `epoch` value, which is not unique.

RECOMMENDATION

Update the natspec to correctly describe the parameters emitted in the `Locked` Event.

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/f91208575671326eabd79c94663eaf306ece72f4>

INFORMATIONAL I-05

RESOLVED

Readme Addresses setToggleLock Function

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/push-chain-migration-push-migration-team1/blob/main/README.md
REVIEW	Main Review

DESCRIPTION

The `README` file key features states:

Safety toggles to prevent/allow locking – Owner Controlled

However, the `setToggleLock` function was removed, and the `lock` is now based on the `pause/unpause` functionality

RECOMMENDATION

Make sure README clearly addresses the features contained in the smart contracts.

RESOLUTION

Status: Resolved.

<https://github.com/pushchain/push-chain-migration/pull/22/commits/247c4790f6c7255feb3b36812f0da5760fd0deff>

INFORMATIONAL **I-06**

ACKNOWLEDGED

Redundant Tree Construction In proofArray.js

CATEGORY Best Practices

LOCATION <https://github.com/push-protocol/push-chain-migration/blob/a056ca2ce60fa54f98fe021004a2676bac7a8c3f/script/testUtils/proofArray.js#L4-L5>

REVIEW Main Review

— DESCRIPTION

A tree is constructed for each user, and then again twice for both `getProof` and `verify` in `proofArray.js`.

This is incredibly inefficient, as the tree remains the same for each user.

— RECOMMENDATION

Consider constructing the tree once and passing it as an object instead.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Unnecessary Indexed Event Params

CATEGORY	Events
LOCATION	https://github.com/pushchain/push-chain-migration/blob/21-audit-fixes/src/MigrationLocker.sol#L41
REVIEW	Remediation Review

DESCRIPTION

```
event NewEpoch(uint256 indexed epoch, uint256 indexed startBlock);
```

This event will only be emitted once per epoch, which will likely be a different block number.

RECOMMENDATION

Remove the indexed event params.

RESOLUTION

Status: Acknowledged.

<https://github.com/pushchain/push-chain-migration/commit/883db319eff2e75dbd0fae8ebcf0f8a120b98909>

INFORMATIONAL I-01

ACKNOWLEDGED

Offchain Balance Checks Affected By Donations

CATEGORY	Logical Error
LOCATION	push-chain-migration/script/utils/fetchAndStoreEvents.js at 21-audit-fixes · push-chain-migration
REVIEW	Remediation Review

DESCRIPTION

Push has added strict balance validations in the `fetch` script.

However, if even 1 wei is accidentally or deliberately sent (donated) to the contract, these validations may cause the script to revert or throw an error.

RECOMMENDATION

Be aware of this edge case during debugging or triage.

If a `throw` is observed, consider the possibility of donations as a potential cause.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.